

Систематско тестирање на FencingCoach со примена на тестирање базирано на својства, мутациско тестирање и тестирање од крај до крај

Студент / индекс: [ПОПОЛНИ ИМЕ И БРОЈ НА ИНДЕКС]

Тим и придонеси: [ПОПОЛНИ ИЛИ ПОТВРДИ ЕДЕН СТУДЕНТ]

Предмет: Софтверски квалитет и тестирање

Линк до репозиториум: [ПОПОЛНИ ПО ДОЗВОЛЕНО ОБЈАВУВАЊЕ]

Линк за пријава / предавање: [ПОПОЛНИ]

Планиран рок: 28 септември 2026, 23:59, Europe/Skopje

Состојба: локалните тестови и мерења се извршени; личните податоци, пристапниот линк и академското предавање остануваат за пополнување. Нема спојување во main, push или предавање.

1. Цел и опсег

FencingCoach is a single-user fencing coach app with a Next.js frontend, FastAPI backend, PostgreSQL, Garmin sync, and optional LLM. The question is how an independently built example suite compares with that same suite after generated properties and then meaningful mutation-guided regressions. The existing backend/tests/ suite is a separate regression suite; none of its cases enters the project measurements.

```
Chromium / Next.js —HTTP→ FastAPI —SQL→ disposable PostgreSQL 16
                        |
                        └─> substituted nutrition agent / USDA boundary

Pure tests: schedule + Garmin extractors
Migrated-DB service tests: nutrition targets + day selection
Mutation population: schedule.py + garmin_extract.py only
```

The test stack has its own database, credentials, volume, loopback ports, and Compose project. The reset helper verifies the destination before truncation. It contains only a synthetic athlete and excludes private .env, real Garmin/model credentials, and sync workers. The real API, background estimate job, database and production frontend run. Live provider quality and medical validity are outside scope.

2. Independent baseline

The [requirement ledger](#) records SCH-01/02, GAR-01/02/03, NUT-01/02/03/04, DAY-01/02, and API-01..05. Expected results were selected at agreed public seams: service functions for calculations, real HTTP plus migrated PostgreSQL for persistence, and Chromium for three user workflows. These seams were confirmed in SPEC.md.

Schedule examples partition valid/invalid tokens, lengths 0/6/7/8, mixed case/space, and Monday/Sunday order. Garmin examples partition absent/null/nonnumeric/in-range/out-of-range scalar candidates, zero versus zero HRV, payload-only readings, and preferred/fallback paths. Nutrition uses worked 89 kg fallback and 60/80 kg profiles, phase/day-type changes, macro-energy agreement, 4 versus 5 usable Garmin days, and the -14/-15/today rolling-window boundaries. A 3,600 kcal history on a rest day makes maintenance changes visible rather than masked by macro floors. A schedule control-flow example traverses token count, validity, mapping and weekday lookup; branch coverage is not edge-pair or all-definition-use-path coverage.

The baseline was frozen in local commit d24a34d before properties or mutation regressions. Its [identity record](#), [JUnit](#), [coverage JSON](#), [coverage XML](#), and [coverage HTML](#) preserve the selection and hashes. **24 cases passed**. Across schedule (4/4), Garmin extraction (22/32) and targets (17/24), 43/60 branches were covered (71.7%). This is scoped service evidence, not app-wide coverage.

3. Generated properties

The expanded suite leaves the frozen baseline files unchanged and adds 13 property functions: four schedule, six Garmin, three PostgreSQL-backed nutrition. Repeatable settings use derandomize=True with 100 examples per pure property (1,000 configured across ten functions) and 20+15+15 for database properties (50 configured). These are **1,050 configured generated examples**, distinct from 37 pytest cases. No generated failure

occurred, so no fabricated shrunk counterexample or production calculation fix is claimed. Database properties use `deadline=None` because database round-trip time varies; no health check was suppressed.

The properties cover valid schedule order, normalization/rejection, Garmin trusted values and unrelated-key invariance, precedence, finite macro-consistent targets, nondecreasing protein/carbohydrate with weight, maintenance-window invariance, and date-isolated overrides. Garmin strategies generate bounded dictionaries with omitted and null fields, numeric strings, body-battery point lists, readiness lists, and max-metrics lists; trusted extracted values must remain finite and within the documented range. NaN/infinity are separate in-process robustness inputs, not JSON. Every generated database example calls `guarded reset_and_seed()` inside its body; the history-order property resets a second time before reinserting rows in reverse order. A function-scoped pytest fixture alone would not isolate examples. The [expanded evidence](#) records **37 passed**, 20.80 seconds of pytest time, and 53/60 scoped branches (88.3%) on the same measured service source hash as baseline. [Expanded coverage XML](#) and [HTML](#) are retained.

4. Mutation comparison

Mutmut 3.8.0 is pinned in `uv.lock`. Generated workspaces copy the full app support package and selected tests, and import `mutants/app`. The clean test run and forced-fail instrumentation check passed; a [selected smoke mutant](#) was killed. Three fresh workspaces use two workers, a 300-second limit, the same schedule/Garmin source hash `8fb68ea0a8a4bc021d9489ad76dbcf795622fffee9511a97df5a8467ba31ff34`, and exactly the same 466 mutant IDs/diffs. Stage A recorded revision `d24a34d`, while B/C recorded `8c5148a`; those later commits changed tests and frontend code but not either mutated source file. [A](#), [B](#) and [C](#) retain configurations, all-ID outcomes, generated mutant source and durations.

The comparison deliberately selects **only schedule/Garmin pure tests** from the frozen baseline. Nutrition target examples/properties, API/browser cases, and legacy tests are measured separately and excluded from mutation scores. This restriction was declared before all runs in the workspace builder, aligning tests with the two-file source allowlist.

Stage	Killed	Survived	No tests	Timeout/error/skipped	Unadjusted resolved score
A: schedule/Garmin baseline	248	218	0	0	248/466 = 53.2%
B: A + schedule/Garmin properties	265	201	0	0	265/466 = 56.9%
C: B + GAR-01/03 regressions	287	179	0	0	287/466 = 61.6%

The resolved score is $\text{killed} \div (\text{killed} + \text{survived} + \text{no-tests})$ across 466 distinct executable IDs. No equivalent adjustment is claimed. B kills 17 additional mutations in calorie, body-battery, readiness, and $\text{VO}_2\text{-max}$ extraction after generated list and omission shapes were added. C kills 22 more: a nested sleep path, body-battery precedence, and raw-payload retention are meaningful observed behavior missing from weaker tests. `x_extract_sleep__mutmut_11`, for example, removes the preferred `dailySleepDT0.sleepTimeSeconds` path; the new 7,200/3,600-second conflict test catches the changed 2 h to 1 h result. `x_build__mutmut_2` drops a missing metric's payload; the regression checks that diagnostic payload. `x_extract_training_status__mutmut_21` changes only detail text for empty payloads while leaving status and scalar meaning intact, so no brittle exact-message test was added just for score. Other survivors remain individually unresolved; none is called equivalent without proof. See [mutation analysis](#) for representative diffs/dispositions.

5. Live API and browser workflows

The [isolated live-stack run](#) passed **five API/database checks and four Chromium checks**, with zero retries or flaky passes. [API JUnit](#) and [Playwright JSON](#) are distinct from mutation evidence. Each API case resets synthetic committed rows; each browser case resets rows and gets a fresh context. Browser noon UTC and backend athlete date are fixed. Polling uses bounded conditions; a real server, rather than a synchronous test client, observes background-job completion.

The day-type API replaces one date twice, verifies one persisted row, rejects invalid input, leaves an adjacent date alone, and clears to Auto. Chromium changes type, sees manual status and updated targets, reloads, then returns to Auto. The reviewed-meal checks establish that estimation alone saves zero meals; edited values save exactly one meal, survive reload and update per-date totals; deletion persists. API checks also filter dates. A controlled FAIL: estimate reaches terminal error, unknown IDs return 404, the browser exits busy state with no log, and a following success reaches review on the same page. Workout API checks replace/read/replace a Tuesday session, validate sets/ reps/load/RPE bounds, leave Thursday unchanged, and clear with an empty list. No workout editor was built.

6. Findings, course techniques, and limits

[Defects](#) records MIG-01, a fresh-install Alembic revision-column overflow fixed transactionally, and UI-01, the browser-discovered absence of accessible names on the day-type selector and review fields. The red trace/screenshot, minimal fix, and passing rerun are retained. Neither is attributed to mutation. No nutrition calculation defect was confirmed. Intended maintain goal behavior, mixed strength/fencing on a rest day, and Garmin numeric-string/boolean coercion remain open policies; current behavior is characterized, not changed.

The supplied [ch06-4-ispinclassexercise-2.pdf](#) describes input-domain modeling; our partitions apply that method. [graph_coverage_example.pdf](#) separates edge-pair requirements, while [skit_06_graph_coverage.pdf](#) discusses control-flow and definition/use; our numbers measure only branches. [skit_01_unittesting-2.pdf](#) and [skit_02_unittesting2-2.pdf](#) address unit tests and time bounds; pure properties and bounded polling use different seams. [api_testing_with_postman.pdf](#) discusses requests, collections and environments; live HTTP tests automate requests against an isolated environment. [selenium_ui_testing.pdf](#) and [selenium_1.pdf/selenium_2.pdf](#) cover browser interaction and locators; Playwright uses role-based selectors in Chromium. [mockito_framework_3a_тестирање.pdf](#) motivates substituting collaborators; only external nutrition/USDA boundaries are replaced. These are supplied local filenames, not invented publication links.

7. Reproduction and acceptance

Install `uv sync --frozen`; run `npm ci` and `npx playwright install chromium` in frontend. Docker Compose is required for database/browser work. Run sequentially from the repository root:

```
bash testing/scripts/run_backend_tests.sh baseline
bash testing/scripts/run_backend_tests.sh expanded
bash testing/scripts/run_e2e.sh
bash testing/scripts/run_mutation_stage.sh A
bash testing/scripts/run_mutation_stage.sh B
bash testing/scripts/run_mutation_stage.sh C
```

The pinned local toolchain was Python 3.12.3, uv 0.11.6, pytest 8.3.4, Hypothesis 6.168.1, coverage 7.16.1, pytest-cov 7.1.0, Mutmut 3.8.0, Playwright 1.63.0, Node 22.20.0, Docker Compose 2.34.0 and PostgreSQL 16. The first baseline wrapper took 281 seconds including image download/build; pytest and mutation durations are recorded separately. [testing/README.md](#) lists ordinary repository checks and environment controls.

Separate gates: **248 legacy tests**, Ruff, mypy, TypeScript and the frontend build passed. `npm run lint` opens interactive setup and is **not a pass**. See [quality gates](#).

Acceptance: Core local service, mutation, API and browser measurements are complete. The editable report, PDF and [demonstration script](#) are local deliverables. Before academic submission, fill identifiers/contributions and real links, check course eligibility and final revision accessibility, then use the actual course process. No push, merge or submission has been performed.